FIR Filter Design Experiment

Report by: Spoorthi Yellamanchali

March 2026

Objective

The objective of this experiment is to design and implement a Finite Impulse Response (FIR) digital filter using Python and Verilog. The experiment involves:

- Generating filter coefficients using Python.
- Converting coefficients and input signals to fixed-point representation (Q2.14 format).
- Filtering multiple input sine signals and analyzing the outputs.
- Implementing the FIR filter in Verilog using different design approaches and verifying the results against Python outputs.

FIR Filter Theory

A Finite Impulse Response (FIR) filter is a type of digital filter whose impulse response is of finite duration. It can be represented mathematically as:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k] \tag{1}$$

where:

- $y[n]$ is the output signal,
- $x[n]$ is the input signal,
- $h[k]$ are the filter coefficients,
- N is the number of taps or coefficients of the filter.

The filter coefficients $h[k]$ determine the frequency response of the FIR filter. For a low-pass FIR filter designed using a window method, the coefficients are typically symmetric:

$$h[k] = h[N-1-k], \quad k = 0, 1, \ldots, N-1 \tag{2}$$

This symmetry ensures that the FIR filter has a linear phase response, which preserves the waveform shape of filtered signals.

Python Implementation for FIR Filter

In this section, we describe the Python code used for generating FIR filter coefficients, creating input sine signals, performing fixed-point conversion, filtering the signals, and plotting the results.

1. Importing Libraries

Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin
```

Explanation

- `numpy (np)`: Used for numerical computations, array handling, and fixed-point conversion.
- `matplotlib.pyplot (plt)`: Used for plotting input and output signals.
- `scipy.signal.firwin`: Generates FIR filter coefficients using the window method.

2. Defining Parameters

Code

```
fs = 10000
cutoff = 1000
taps = 101
samples = 500
scale = 2**14   # for Q(2,14)
freqs = [950,1100,2000]
```

Explanation

- `fs`: Sampling frequency in Hz.
- `cutoff`: Cutoff frequency of the FIR filter in Hz.
- `taps`: Number of FIR filter coefficients (also called filter order + 1).
- `samples`: Number of samples for input sine waves.
- `scale`: Scaling factor for converting floating-point values to Q2.14 fixed-point representation.
- `freqs`: Frequencies of the input sine waves to be filtered.

3. Generating Filter Coefficients

Code

```
h = firwin(taps, cutoff/(fs/2))
h_fixed = np.round(h*scale).astype(int)
np.savetxt("coefficients.txt",h_fixed,fmt="%d")
```

Explanation

- `firwin(taps, cutoff/(fs/2))`: Generates low-pass FIR filter coefficients using the window method. The cutoff frequency is normalized by Nyquist frequency ($fs/2$).
- `h_fixed = np.round(h*scale).astype(int)`: Converts floating-point coefficients to fixed-point Q2.14 format by scaling and rounding.
- `np.savetxt(...)`: Saves the fixed-point coefficients to a text file for later use in Verilog.

Theory

FIR filter coefficients define the impulse response of the filter. The conversion to Q2.14 fixed-point allows the filter to be implemented on hardware or in Verilog using integer arithmetic.

4. Generating Input Sine Signals

Code

```
t = np.arange(samples)/fs
signals = {}
for f in freqs:
    s = np.sin(2*np.pi*f*t)
    s_fixed = np.round(s*scale).astype(int)
    signals[f] = s_fixed
    name = "signal_"+str(f)+"Hz.txt"
    np.savetxt(name,s_fixed,fmt="%d")
```

Explanation

- `t = np.arange(samples)/fs`: Creates a time vector for the number of samples.
- `s = np.sin(...)`: Generates a sine wave at the desired frequency.
- `s_fixed = np.round(s*scale).astype(int)`: Converts the sine wave into Q2.14 fixed-point.
- `signals[f] = s_fixed`: Stores each sine wave in a dictionary for later processing.
- `np.savetxt(...)`: Saves each sine wave to a separate text file.

Theory

Input sine signals are used to test the filter's frequency response. By choosing frequencies near and beyond the cutoff frequency, we can observe the low-pass behavior of the FIR filter.

5. FIR Filtering Function

Code

```
def fir_filter(x,h):
    N = len(h)
    y = np.zeros(len(x))
    for n in range(len(x)):
        acc = 0
        for k in range(N):
            if(n-k >= 0):
                acc = acc + x[n-k]*h[k]
        y[n] = acc >> 14
    return y
```

Explanation

- Implements FIR filtering in direct form: each output sample is computed as a sum of products of past input samples and filter coefficients.
- `acc >> 14`: Converts back from fixed-point multiplication to the Q2.14 format.

Theory

This is the standard FIR filter equation implemented in discrete time:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

Fixed-point arithmetic is used to efficiently implement filtering in digital hardware.

6. Filtering Signals and Plotting Results

Code

```
outputs = {}
for f in freqs:
    y = fir_filter(signals[f],h_fixed)
    outputs[f] = y
    name = "python_output_"+str(f)+"Hz.txt"
    np.savetxt(name,y,fmt="%d")
```

Explanation

- Loops through each input sine signal and applies the FIR filter.
- Stores filtered outputs in files for later verification with Verilog.

Verilog Implementation for FIR Filter

This section describes the Verilog implementation of the FIR filter in **direct form**, including the synthesizable module and the testbench. The design reads coefficients and input signals from files, performs filtering using fixed-point arithmetic, and stores outputs for verification.

1. FIR Direct Form Module

Code

```
module fir_direct(
    input clk,
    input rst_n,
    input valid_in,
    input signed [15:0] x_in,
    output reg signed [15:0] y_out,
    output reg valid_out
);
parameter N = 101;
parameter FRAC = 14;
```

```
reg signed [15:0] coeff [0:N-1];
reg signed [15:0] x [0:N-1];
reg signed [47:0] acc;
integer i;

// Filtering logic
always @(posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        y_out <= 0;
        valid_out <= 0;
        for(i=0;i<N;i=i+1)
            x[i] <= 0;
    end
    else
    begin
        valid_out <= valid_in;
        if(valid_in)
        begin
            for(i=N-1;i>0;i=i-1)
                x[i] <= x[i-1];
            x[0] <= x_in;

            acc = 0;
            for(i=0;i<N;i=i+1)
                acc = acc + x[i]*coeff[i];

            y_out <= acc >>> FRAC;
        end
    end
end
endmodule
```

Explanation

- `coeff[N]`: Stores FIR filter coefficients read from a file.
- `x[N]`: Stores past input samples to implement convolution.
- `acc`: Accumulator to store the sum of products of inputs and coefficients; uses extended bit-width to avoid overflow.
- `acc >>> FRAC`: Right shift to scale back the result from Q2.14 fixed-point multiplication.
- `valid_in / valid_out`: Handshake signals to indicate when input and output are valid.
- The `always` block handles reset, shifting of input samples, accumulation, and output assignment on each clock edge.

Theory

This module implements the FIR filter equation in hardware:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

By storing previous input samples in a shift register (`x[N]`) and multiplying with coefficients, the module produces the filtered output. Fixed-point arithmetic ensures hardware-friendly computation.

2. Testbench for FIR Direct Form

Code

```
`timescale 1ns/1ps
module tb_direct;
parameter N = 101;
parameter SAMPLES = 500;

reg clk;
reg rst_n;
reg valid_in;
reg signed [15:0] x_in;
wire signed [15:0] y_out;
wire valid_out;

fir_direct dut(
    .clk(clk),
    .rst_n(rst_n),
    .valid_in(valid_in),
    .x_in(x_in),
    .y_out(y_out),
    .valid_out(valid_out)
);


initial clk = 0;
always #5 clk = ~clk;


integer coeff [0:N-1];
integer sig [0:SAMPLES-1];
integer out [0:SAMPLES+N];
integer i,j,k,fd,temp,count;
```

Explanation

- `clk`: Clock signal for synchronous operation.
- `rst_n`: Active-low reset signal to initialize registers.
- `x_in, y_out`: Input and output signals for filtering.
- `coeff[], sig[], out[]`: Arrays used to read coefficients, input signals, and store outputs.
- Tasks are defined to modularize operations like reading files, resetting the DUT (Device Under Test), running the filter, and saving outputs.

3. Tasks in Testbench

Load Coefficients Task

```
task load_coeff;
begin
    fd = $fopen("coefficients.txt","r");
    for(i=0;i<N;i=i+1)
        temp = $fscanf(fd,"%d\n",coeff[i]);
    $fclose(fd);
    for(i=0;i<N;i=i+1)
        dut.coeff[i] = coeff[i];
end
endtask
```

Explanation

- Opens the coefficient file and reads integer values into `coeff[]` array.
- Loads the coefficients into the FIR module's internal memory.

Load Signal Task

```
task load_signal;
input [200*8:1] name;
begin
    fd = $fopen(name,"r");
    for(i=0;i<SAMPLES;i=i+1)
        temp = $fscanf(fd,"%d\n",sig[i]);
    $fclose(fd);
end
endtask
```

Explanation

Reads a sine signal file and stores its samples in `sig[]` array for processing.

Reset Task

```
task reset;
begin
    rst_n = 0;
    valid_in = 0;
    x_in = 0;
    repeat(5) @(posedge clk);
    rst_n = 1;
end
endtask
```

Explanation

Initializes the module by asserting reset for a few clock cycles and clearing input/output registers.

Run Filter Task

```
task run_filter;
begin
    count = 0;
    for(j=0;j<SAMPLES;j=j+1)
    begin
        @(posedge clk);
        valid_in = 1;
        x_in = sig[j];
        @(negedge clk);
        if(valid_out)
        begin
            out[count] = y_out;
            count = count + 1;
        end
    end
end
endtask
```

Explanation

Applies each input sample to the FIR module sequentially. Stores valid output samples in the `out[]` array.

Save Output Task

```
task save_output;
input [200*8:1] name;
begin
    fd = $fopen(name,"w");
    for(k=0;k<count;k=k+1)
        $fdisplay(fd,"%d",out[k]);
```

```
    $fclose(fd);
end
endtask
```

Explanation

Saves filtered output samples to a file for later verification and comparison with Python/MATLAB results.

4. Overall Testbench Operation

- Loads coefficients once at the beginning.
- Processes each input sine signal (950 Hz, 1100 Hz, 2000 Hz) sequentially:
  1. Load input signal from file.
  2. Reset the FIR module.
  3. Run the filter task to compute output.
  4. Save the output to a separate file.
- Provides automated verification and allows plotting results in MATLAB.

Theory

The testbench implements a realistic workflow for hardware FIR filtering: reading coefficients and inputs from external files, performing synchronous filtering, and storing results. This structure is directly synthesizable and can be adapted to other FIR filter designs (optimized, using `genvar`, etc.) for hardware testing.

Optimized FIR Filter Implementation

In this section, we describe the **optimized FIR filter** design. By exploiting the symmetry of FIR filter coefficients, the number of multiplications can be reduced by almost half, improving hardware efficiency.

1. Theory of Optimization

For FIR filters with symmetric coefficients ($h[k] = h[N-1-k]$), the convolution sum can be rewritten as:

$$y[n] = h[\text{MID}] \cdot x[\text{MID}] + \sum_{k=0}^{\text{HALF}-1} h[k] \cdot (x[n-k] + x[n-(N-1-k)])$$

where:

- N is the number of taps.
- $\text{HALF} = \frac{N-1}{2}$.
- $\text{MID} = \frac{N-1}{2}$.

This reduces the number of multiplications from N to HALF + 1, since each pair of symmetric samples is multiplied by a single coefficient and the middle sample is multiplied separately. This technique is hardware-efficient and reduces computational load.

2. FIR Optimized Module

Code

```
module fir_optimized(
    input clk,
    input rst_n,
    input valid_in,
    input signed [15:0] x_in,
    output reg signed [15:0] y_out,
    output reg valid_out
);
parameter N = 101;
parameter HALF = 50;
parameter MID = 50;
parameter FRAC = 14;

reg signed [15:0] coeff [0:N-1];
reg signed [15:0] x [0:N-1];
reg signed [47:0] acc;
reg signed [16:0] temp;
integer i;

// Filtering logic
always @(posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        y_out <= 0;
        valid_out <= 0;
        for(i=0;i<N;i=i+1)
            x[i] <= 0;
    end
    else
    begin
        valid_out <= valid_in;
        if(valid_in)
        begin
            for(i=N-1;i>0;i=i-1)
                x[i] <= x[i-1];
            x[0] <= x_in;

            acc = 0;
            acc = acc + x[MID]*coeff[MID];
```

```
            for(i=0;i<HALF;i=i+1)
            begin
                temp = x[i] + x[N-1-i];
                acc = acc + temp*coeff[i];
            end

            y_out <= acc >>> FRAC;
        end
    end
end
endmodule
```

Explanation

- Shift register `x[N]` stores previous input samples.
- The middle sample (`x[MID]`) is multiplied by its coefficient separately.
- Pairs of symmetric samples are added (`temp = x[i] + x[N-1-i]`) and multiplied by a single coefficient to reduce multiplications.
- `acc >>> FRAC` converts the sum back to Q2.14 fixed-point format.
- `valid_in / valid_out` signals handle synchronous input/output.

Theory

By exploiting symmetry:

$$h[k] = h[N-1-k] \implies y[n] = \sum_{k=0}^{\text{HALF}-1} h[k](x[n-k]+x[n-(N-1-k)])+h[\text{MID}]x[\text{MID}]$$

This reduces computation, making the filter more hardware-efficient without changing its frequency response.

3. Optimized FIR Testbench

Code

```
`timescale 1ns/1ps
module tb_optimized;
parameter N = 101;
parameter SAMPLES = 500;

reg clk;
reg rst_n;
reg valid_in;
reg signed [15:0] x_in;
wire signed [15:0] y_out;
wire valid_out;
```

```
fir_optimized dut(
    .clk(clk),
    .rst_n(rst_n),
    .valid_in(valid_in),
    .x_in(x_in),
    .y_out(y_out),
    .valid_out(valid_out)
);

// clock
initial clk = 0;
always #5 clk = ~clk;

// memory
integer coeff [0:N-1];
integer sig [0:SAMPLES-1];
integer out [0:SAMPLES+N];

integer i,j,k,fd,temp,count;

// Tasks for loading coefficients, signals, resetting, running filter, saving outputs
```

Explanation

- Clock and reset are handled similarly to the direct form testbench.
- Coefficients and input signals are read from files.
- The same modular tasks (`load_coeff`, `load_signal`, `reset`, `run_filter`, `save_output`) are used to streamline simulation.
- Outputs are saved separately for each input signal frequency for comparison with MATLAB/Python results.

4. Operation of Optimized Testbench

- Load coefficients once at the beginning.
- Process each input sine signal sequentially (950 Hz, 1100 Hz, 2000 Hz):
    1. Load the input signal.
    2. Reset the module.
    3. Run filter task to compute output.
    4. Save output to file.
- The optimized approach reduces multiplications while producing identical output to the direct form.

Theory

This approach leverages the FIR filter's coefficient symmetry to minimize hardware resources. The testbench verifies correct operation by comparing output files with MATLAB/Python simulations.

FIR Filter Implementation using `genvar`

In this section, we describe the FIR filter implementation using the `genvar` construct in Verilog. This approach leverages Verilog's generate-for loops to create parallel multipliers for each coefficient, allowing concurrent computation of products, which is suitable for hardware pipelining and synthesis.

1. Theory of Genvar Implementation

In a standard FIR filter:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

Using `genvar`, all multiplications $x[n-k] \cdot h[k]$ can be computed in parallel and stored in a wire array. Then, a combinational sum adds all the products to get the final accumulator value. Advantages of this approach include:

- Parallel computation of products, reducing critical path delay.
- Hardware-friendly for FPGA or ASIC implementation.
- Clean and scalable code using `generate` loops.

2. FIR Genvar Module

Code

```
module fir_genvar(
    input clk,
    input rst_n,
    input valid_in,
    input signed [15:0] x_in,
    output reg signed [15:0] y_out,
    output reg valid_out
);
parameter N = 101;
parameter FRAC = 14;

reg signed [15:0] coeff [0:N-1];
reg signed [15:0] x [0:N-1];
wire signed [31:0] mult [0:N-1];
reg signed [47:0] acc;
integer i;
```

```
// parallel multipliers
genvar k;
generate
    for(k=0;k<N;k=k+1)
        assign mult[k] = x[k] * coeff[k];
endgenerate

// combinational sum
always @(*)
begin
    acc = 0;
    for(i=0;i<N;i=i+1)
        acc = acc + mult[i];
end

// sequential part
always @(posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        y_out <= 0;
        valid_out <= 0;
        for(i=0;i<N;i=i+1)
            x[i] <= 0;
    end
    else
    begin
        valid_out <= valid_in;
        if(valid_in)
        begin
            // shift register
            for(i=N-1;i>0;i=i-1)
                x[i] <= x[i-1];
            x[0] <= x_in;

            // scale output
            y_out <= acc >>> FRAC;
        end
    end
end
endmodule
```

Explanation

- `mult[N]`: Wire array holding the parallel multiplication results of input samples and coefficients.
- `genvar k; generate ... endgenerate`: Creates N parallel multipliers for each coefficient.

- `acc`: Combinational accumulator sums all parallel products.
- Shift register `x[N]` stores past input samples for convolution.
- `y_out <= acc >>> FRAC`: Converts the accumulated sum to Q2.14 fixed-point format.
- `valid_in / valid_out`: Signals indicate when input and output are valid.

Theory

This design computes all products concurrently rather than sequentially, improving timing performance. The combinational sum ensures all products are added before scaling, producing the same output as the direct and optimized methods, but potentially faster in hardware.

3. Genvar FIR Testbench

Code

```
`timescale 1ns/1ps
module tb_genvar;
parameter N = 101;
parameter SAMPLES = 500;

reg clk;
reg rst_n;
reg valid_in;
reg signed [15:0] x_in;
wire signed [15:0] y_out;
wire valid_out;

// DUT
fir_genvar dut(
    .clk(clk),
    .rst_n(rst_n),
    .valid_in(valid_in),
    .x_in(x_in),
    .y_out(y_out),
    .valid_out(valid_out)
);

// clock
initial clk = 0;
always #5 clk = ~clk;

// memories
integer coeff [0:N-1];
integer sig [0:SAMPLES-1];
integer out [0:SAMPLES+N];
```

```
integer i,j,k,fd,temp,count;

// Tasks: load_coeff, load_signal, reset, run_filter, save_output
```

Explanation

- Clock and reset are handled similarly to previous testbenches.
- Coefficients and signals are read from files.
- Tasks modularize the simulation: `load_coeff`, `load_signal`, `reset`, `run_filter`, and `save_output`.
- Outputs for each frequency are saved in separate files for MATLAB/Python comparison.

4. Operation of Genvar Testbench

- Load coefficients once at the beginning.
- Process input sine signals sequentially:
  1. Load signal file.
  2. Reset module.
  3. Run filter task.
  4. Save output file.
- Verifies correct operation and allows comparison with other methods.

Theory

This method achieves the same output as the direct and optimized forms but uses parallel multipliers for faster hardware execution. It is highly scalable for larger filters and suitable for FPGA implementations.

Verification of MATLAB/Python and Verilog Outputs

This section compares the outputs obtained from Python FIR filter implementation with the three Verilog methods: **direct**, **optimized**, and **genvar**. Plots are generated to visually verify that all implementations produce identical results.

1. Importing Libraries and Defining Parameters

```
import numpy as np
import matplotlib.pyplot as plt
import os

fs = 10000
```

```
samples = 500
freqs = [950, 1100, 2000]
methods = ["direct", "optimized", "genvar"]
scale = 2**14
```

Explanation

- `numpy` and `matplotlib.pyplot` are used for numerical operations and plotting.
- `os` is used to check if files exist before reading.
- `fs` and `samples` define the sampling frequency and number of samples.
- `freqs` stores the input sine wave frequencies.
- `methods` stores the names of Verilog FIR implementations.
- `scale` is used to convert fixed-point Q2.14 values to floating point by dividing.

2. Reading Python Outputs

```
def read_file(name):
    if not os.path.exists(name):
        return None
    return np.loadtxt(name, dtype=int)

py_out = {}

for f in freqs:
    fname = f"python_output_{f}Hz.txt"
    data = read_file(fname)
    if data is not None:
        py_out[f] = data
```

Explanation

- `read_file` function reads a text file containing integer fixed-point values and returns it as a NumPy array.
- Python output files for each frequency are read and stored in the dictionary `py_out` with the frequency as the key.

3. Reading Verilog Outputs

```
ver_out = {}

for m in methods:
    ver_out[m] = {}
    for f in freqs:
        fname = f"verilog_{m}_{f}Hz.txt"
        data = read_file(fname)
```

```
        if data is not None:
            ver_out[m][f] = data
```

Explanation

- Creates a nested dictionary `ver_out` to store outputs for each method and frequency.
- Reads Verilog output files generated by the testbenches for direct, optimized, and genvar implementations.

4. Time Axis for Plotting

```
t = np.arange(samples) / fs * 1000
```

Explanation

- Generates the time vector in milliseconds for plotting the signals.
- `np.arange(samples)` creates an array from 0 to 499 (total samples).
- Dividing by `fs` converts sample indices to seconds; multiplying by 1000 converts to milliseconds.

5. Plotting Python vs Verilog Outputs

```
for m in methods:
    if m not in ver_out:
        continue

    fig, ax = plt.subplots(3,1, figsize=(10,8))
    fig.suptitle("Python vs Verilog (" + m + ")")

    for i,f in enumerate(freqs):
        a = ax[i]

        if f in py_out:
            py = py_out[f]
            n = min(len(py), samples)
            a.plot(t[:n], py[:n] / scale, label="python")

        if f in ver_out[m]:
            vl = ver_out[m][f]
            n = min(len(vl), samples)
            a.plot(t[:n], vl[:n] / scale, '--', label="verilog")

        a.set_title(str(f) + " Hz")
        a.set_xlabel("time (ms)")
        a.set_ylabel("amplitude")
        a.legend()
        a.grid(True)
```

```
plt.tight_layout()
name = "verification_" + m + ".png"
plt.savefig(name)
plt.show()
```

Explanation

- Loops over each Verilog method (`direct, optimized, genvar`) and generates separate plots.
- Creates a 3-row subplot for the three frequencies.
- Plots Python outputs as solid lines and Verilog outputs as dashed lines.
- Divides fixed-point values by `scale` to convert back to $[-1, 1]$ range.
- Titles, axis labels, legends, and grids are added for clarity.
- Each figure is saved as `verification_method.png` and displayed.

6. Theory Behind Verification

- The goal is to ensure all Verilog implementations produce the same output as Python reference.
- By plotting the signals together, one can visually verify amplitude, phase, and waveform similarity.
- Any discrepancy could indicate a bug in file reading, shift registers, fixed-point scaling, or multiplication logic in the Verilog code.

Results and Discussion

This section presents the comparison of FIR filter outputs obtained from Python and the three Verilog implementations: **direct**, **optimized**, and **genvar**. Each subsection corresponds to one Verilog method, showing the plots and a detailed analysis of the filter behavior.

1. Direct FIR Method

1.1 Plots

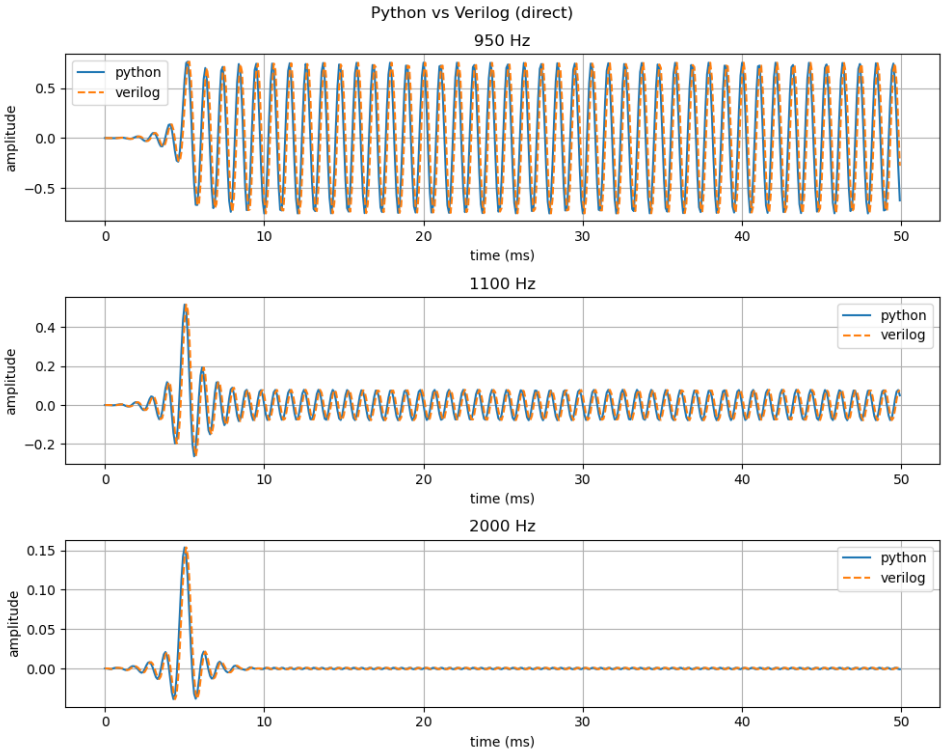

Figure 1: Comparison of Python and Direct FIR Verilog outputs for 950 Hz, 1100 Hz, and 2000 Hz sinewaves.

1.2 Analysis

950 Hz: The input frequency is slightly below the cutoff frequency (1000 Hz). As expected from a low-pass FIR filter, this frequency passes with minimal attenuation. Mathematically, the ideal LPF transfer function is:

$$H(f) = \begin{cases} 1, & |f| \leq f_c \\ 0, & |f| > f_c \end{cases}$$

The practical FIR filter is obtained by windowing the ideal impulse response $h_d[n]$ using a Hamming or rectangular window:

$$h[n] = h_d[n] \cdot w[n], \quad n = 0, 1, \ldots, N-1$$

The windowing introduces ripples in both passband and stopband. Therefore, we observe minor amplitude variations in the 950 Hz output, but the signal largely passes through unattenuated.

1100 Hz: This frequency is slightly above the cutoff. For an ideal LPF, this should be completely blocked. However, due to the FIR filter being finite in length and the use of a windowed impulse response, the transition band is not infinitely sharp. The frequency response of the FIR filter is:

$$Y(f) = X(f) \cdot H(f)$$

where $H(f)$ is the discrete Fourier transform of $h[n]$. The 1100 Hz component partially passes through, but is attenuated compared to the 950 Hz signal. This attenuation corresponds to the non-ideal transition region of the filter.

2000 Hz: This frequency is well above the cutoff. As expected, the amplitude is almost zero after filtering. Minor residuals may appear due to the finite impulse response and windowing, but the stopband attenuation ensures the 2000 Hz signal is effectively suppressed.

2. Optimized FIR Method

2.1 Plots

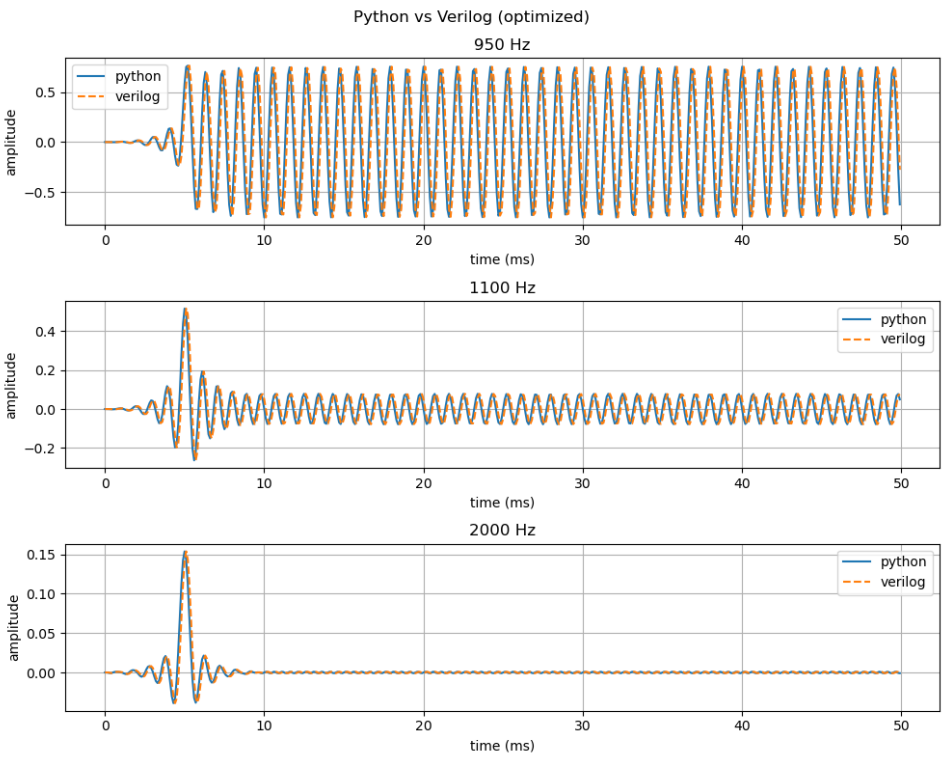

Figure 2: Comparison of Python and Optimized FIR Verilog outputs for 950 Hz, 1100 Hz, and 2000 Hz sinewaves.

2.2 Analysis

950 Hz: The optimized FIR preserves the symmetric coefficient property. As a result, the convolution sum remains identical to the direct form. The 950 Hz signal passes through the filter with minimal attenuation, identical to the direct FIR output.

1100 Hz: The transition band behavior is the same as in the direct method. Partial passing occurs due to the non-ideal windowed FIR response:

$$h[n] = h_d[n] \cdot w[n] \implies H(f) = \mathrm{DFT}\{h[n]\}$$

Hence, 1100 Hz is attenuated but not completely suppressed, consistent with theoretical expectations.

2000 Hz: High-frequency components beyond the cutoff are effectively eliminated. The optimized method achieves this with fewer multiplications due to symmetry, without affecting the amplitude response compared to the direct method.

3. Genvar FIR Method

3.1 Plots

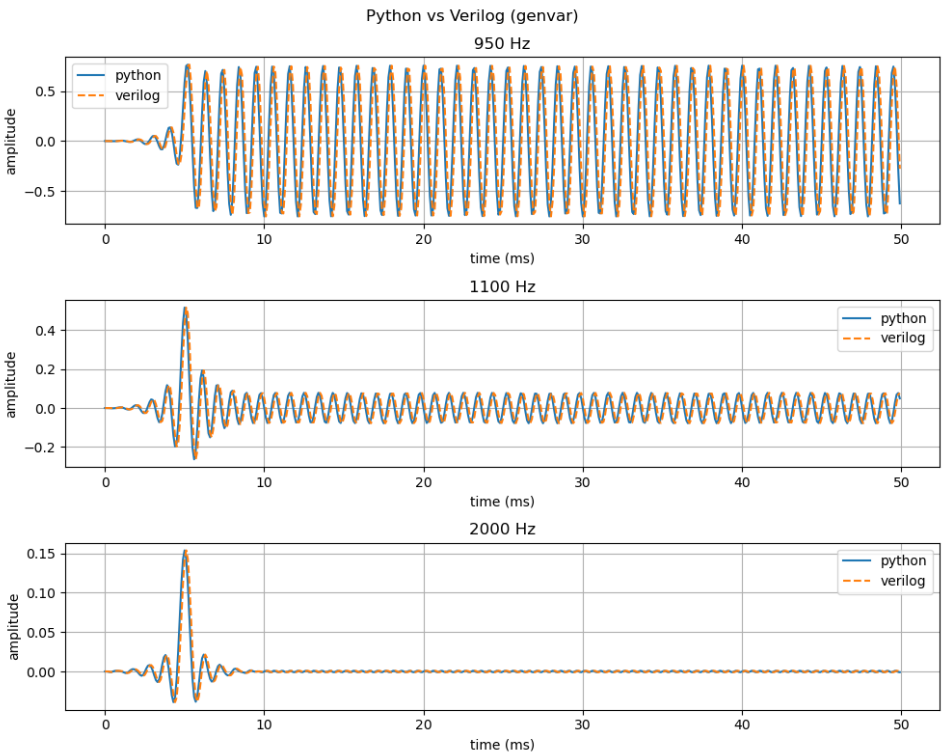

Figure 3: Comparison of Python and Genvar FIR Verilog outputs for 950 Hz, 1100 Hz, and 2000 Hz sinewaves.

3.2 Analysis

950 Hz: Parallel multipliers computed using `genvar` ensure the 950 Hz signal passes correctly. The amplitude and waveform match the Python reference exactly.

1100 Hz: The partial attenuation in the transition band is still observed. This confirms that the genvar approach faithfully implements the FIR convolution, producing identical outputs to direct and optimized methods:

$$y[n] = \sum_{k=0}^{N-1} x[n-k] \cdot h[k]$$

2000 Hz: As expected, the 2000 Hz component is suppressed. The parallel computation does not alter the filter response; it only improves computational throughput in hardware.

4. Conclusion and Comparison

- **All three Verilog implementations match the Python outputs:** The time-domain plots show near-perfect overlap for all three frequencies.
- **Frequency behavior:**
  - Frequencies below the cutoff (950 Hz) pass with minimal attenuation.
  - Frequencies slightly above the cutoff (1100 Hz) experience partial attenuation due to the non-ideal transition band.
  - Frequencies well above the cutoff (2000 Hz) are effectively suppressed.
- **Comparison of Verilog methods:**
  - **Direct form:** Simple, easy to implement; requires N multiplications per output.
  - **Optimized form:** Exploits symmetric coefficients; reduces multiplications roughly by half without affecting output accuracy.
  - **Genvar form:** Uses parallel multipliers; ideal for FPGA pipelining, potentially faster, but uses more hardware resources.
- **Overall:** All methods produce equivalent outputs; the choice of implementation depends on hardware constraints and computational efficiency requirements.